

Taylor Polynomial versus Krylov–Arnoldi for QFLPN Operator Evolution: Accuracy, Stability, and Computational Cost

Adrian Roman

September 12, 2026

Abstract

Large sparse state-evolution problems require matrix-function evaluations without forming dense matrix exponentials. This paper presents a controlled comparison of scaled truncated Taylor propagation and Krylov–Arnoldi projection for the matrix-exponential action $e^{tA}v$, using the same sparse operator, normalized initial vector, arithmetic precision, accuracy targets, and independently computed reference solution. The primary benchmark uses the real tridiagonal operator $A_s = s \operatorname{tridiag}(1, -2, 1)$ with $N = 5,000$, $t = 1$, scales $s \in \{1, 5, 10, 20\}$, and target relative errors 10^{-6} , 10^{-8} , and 10^{-10} . Taylor propagation is controlled by an explicit remainder bound and scaling, whereas Arnoldi constructs a Krylov subspace with modified Gram–Schmidt and reorthogonalization and evaluates a projected exponential. The comparison reports independently measured relative error, median runtime, sparse matrix–vector-product-equivalent cost, Arnoldi residual indicators, and an explicit differential-memory model. At $s = 20$ and target 10^{-10} , scaled Taylor attains a relative error of 2.14×10^{-14} in 19.479 ms using 930 SpMV-equivalents, while the smallest tested Arnoldi configuration satisfying the target has dimension $m = 50$ and a measured runtime of 27.137 ms. A secondary $N = 20,000$ experiment at target 10^{-8} preserves the qualitative ordering for the tested operator family but is not used to infer an asymptotic scaling law. The results therefore support a regime-dependent selection rule rather than universal superiority: the preferred method depends on operator scale, target tolerance, Krylov convergence, and the relative cost of sparse matrix–vector products and orthogonalization. The complete benchmark contract and measured outputs are specified for independent reproduction.

Keywords: QFLPN; matrix exponential; sparse operators; truncated Taylor series; Krylov subspace; Arnoldi method; exponential propagation; numerical stability; reproducible benchmarking

1 Introduction

The numerical evolution of a large sparse state is fundamentally different from the explicit manipulation of a small quantum register. Once the state dimension is large, the central computational question is no longer how to write the operator in dense form, but how to evaluate the required matrix function while preserving accuracy and controlling the cost of repeated sparse matrix–vector products. This issue is particularly important when a transition operator is generated from a structured discrete, fuzzy, or quantum model and the resulting state dimension is too large for dense matrix storage.

A recurring operation in such systems is the action of the matrix exponential,

$$y = e^{tA}v, \tag{1}$$

where $A \in \mathbb{R}^{N \times N}$ or $A \in \mathbb{C}^{N \times N}$ is sparse, v is an initial state, and t is a propagation parameter. Forming e^{tA} explicitly is usually unnecessary and can be prohibitively expensive. Instead, the numerical task is to approximate its action on v using only products with A .

Two broad strategies are especially relevant. A truncated Taylor method evaluates a sequence of sparse matrix–vector products and controls the truncation error through the Taylor remainder. A Krylov–Arnoldi method constructs a low-dimensional subspace containing $v, Av, A^2v, \dots$ and projects the exponential onto that subspace. The latter approach is a classical and well-established technique for large sparse matrix functions, with theoretical error estimates and a long history in exponential propagation [1, 2, 3, 6].

The broader numerical-linear-algebra context includes stability, matrix-function computation, and iterative subspace methods, for which standard references provide complementary foundations [5, 7, 9]. The scientific question of this paper is therefore not whether either method exists, but under which observable computational regimes one should prefer one over the other for a sparse QFLPN-compatible operator. The question is formulated as an accuracy–cost problem: for a prescribed relative-error target, how many sparse operator applications are required, how much additional orthogonalization work is introduced, and what wall-clock behavior is actually observed?

The contribution is structured around five principles. First, the numerical problem is isolated from the semantic QFLPN construction. Second, Taylor and Arnoldi are expressed using the same operator-access model, so the comparison is not distorted by dense matrix formation. Third, the comparison uses a common error metric and an independently computed reference solution. Fourth, the benchmark reports both an operator-application count and wall-clock time, because those quantities need not rank algorithms identically. Fifth, the study distinguishes measured results from general asymptotic claims and from conclusions that require broader matrices or hardware.

The resulting paper is deliberately distinct from a formal QFLPN definition, a finite four-qubit verification, and a sparse CSR/SpMV implementation study. Its scientific object is the matrix-function evaluation layer: how a sparse transition operator is exponentiated in action, how approximation error is controlled, and how an algorithm should be selected from measurable numerical characteristics.

2 Problem Statement and Scientific Boundary

Let A denote a sparse numerical operator induced by a QFLPN state-evolution model after the required semantic and numerical transformations have been fixed. The present paper studies the action

$$y = e^{tA}v, \tag{2}$$

without constructing the dense matrix e^{tA} .

The vector dimension N is a numerical dimension. It must not be confused with the number of qubits q . For a dense pure state of q qubits, the state-vector dimension is 2^q , whereas a sparse numerical experiment can use an arbitrary dimension N determined by the representation of the transition operator. The present benchmark therefore does not claim to simulate 5,000 qubits.

The operator family used for the primary experiment is the Dirichlet one-dimensional second-difference operator,

$$A_s = s \text{ tridiag}(1, -2, 1), \tag{3}$$

with dimension $N = 5,000$ and scale parameter $s \in \{1, 5, 10, 20\}$. This family is intentionally simple enough to make the numerical comparison transparent and sufficiently sparse to expose the computational structure of the matrix-function action. It should be interpreted as a controlled benchmark family, not as a universal model of all QFLPN operators.

The numerical operator layer is deliberately separated from the quantum-semantic layer. Quantum computation and quantum information provide the state-space and operator context for the QFLPN series, while open-system formulations motivate the broader distinction between unitary and non-

unitary evolution; the present benchmark studies only the sparse matrix-exponential action itself [13, 14, 15, 16].

The input vector v is generated from a fixed pseudorandom seed and normalized to unit Euclidean norm. The propagation parameter is $t = 1$. The reference solution is obtained independently using a sparse matrix-exponential action implementation. All reported errors are relative Euclidean errors with respect to that reference.

The paper does not claim quantum advantage, hardware-level quantum execution, physical decoherence performance, or universal superiority of Taylor or Arnoldi methods. It also does not infer large-scale performance from the $N = 5,000$ experiment without an explicit secondary scaling observation.

3 Numerical Formulation

3.1 Taylor approximation

The exponential has the formal series

$$e^{tA}v = \sum_{j=0}^{\infty} \frac{(tA)^j}{j!} v. \quad (4)$$

A degree- p truncated Taylor action is

$$T_p(tA)v = \sum_{j=0}^p \frac{(tA)^j}{j!} v. \quad (5)$$

The recurrence used in the implementation is

$$w_0 = v, \quad w_j = \frac{t}{j} A w_{j-1}, \quad y_p = \sum_{j=0}^p w_j. \quad (6)$$

Each new Taylor term therefore requires one sparse matrix–vector product.

A standard norm bound gives

$$\|e^{tA}v - T_p(tA)v\|_2 \leq e^{|t|\|A\|_2} \frac{(|t|\|A\|_2)^{p+1}}{(p+1)!} \|v\|_2. \quad (7)$$

This bound is useful for establishing an a priori truncation criterion [10, 5], but it can be conservative because it depends on a matrix norm rather than on the actual action of the operator on the particular vector v .

For the present experiment, direct Taylor propagation is combined with scaling. Scaling-and-squaring and matrix-exponential action algorithms provide the broader numerical context for this design choice [10, ?, 12, 5]. The propagation is divided into 2^r substeps,

$$e^{tA}v = \left(e^{(t/2^r)A} \right)^{2^r} v, \quad (8)$$

so that each local Taylor polynomial is evaluated at a smaller argument. The scaling parameter is chosen from a conservative sparse 1-norm estimate. The local tolerance is reduced with the number of scaling stages. The resulting procedure is not claimed to reproduce the full parameter-selection strategy of specialized production-quality exponential-action software; it is an explicitly documented comparison algorithm.

3.2 Krylov–Arnoldi approximation

The m -dimensional Krylov subspace is

$$\mathcal{K}_m(A, v) = \text{span}\{v, Av, A^2v, \dots, A^{m-1}v\}. \quad (9)$$

Arnoldi orthogonalization constructs an orthonormal basis

$$V_m = [v_1, \dots, v_m] \quad (10)$$

with $v_1 = v/\|v\|_2$ and an upper Hessenberg matrix H_m satisfying

$$AV_m = V_m H_m + h_{m+1,m} v_{m+1} e_m^\top. \quad (11)$$

The exponential action is approximated by

$$e^{tA}v \approx \|v\|_2 V_m e^{tH_m} e_1. \quad (12)$$

The large matrix exponential has therefore been replaced by a small dense exponential of order m .

The implementation uses modified Gram–Schmidt followed by a second orthogonalization pass. Re-orthogonalization is important because loss of orthogonality can corrupt the projected representation even when the sparse matrix–vector products themselves are accurate.

3.3 Residual interpretation

The Arnoldi representation exposes an a posteriori residual indicator. For the exponential action, the residual magnitude is controlled by the coupling coefficient $h_{m+1,m}$ and the last component of the small exponential action. A practical indicator has the form

$$\eta_m(t) = |h_{m+1,m}| \left| e_m^\top e^{tH_m} e_1 \right|. \quad (13)$$

This quantity does not replace an independently computed reference error in the present benchmark. Instead, it explains why Krylov dimension can be interpreted as a controllable approximation parameter.

4 Common Cost Model

A direct comparison of wall-clock time alone can conceal why the methods behave differently. We therefore define an operator-application count as the primary algorithm-independent cost proxy.

For scaled Taylor propagation, let p_j denote the number of Taylor terms retained at scaling stage j . The number of sparse operator applications is

$$C_T = \sum_{j=1}^{2^r} p_j. \quad (14)$$

For Arnoldi with final dimension m ,

$$C_A = m. \quad (15)$$

The quantity C is called the *SpMV-equivalent cost*. It is not a complete time model because Arnoldi additionally performs orthogonalization operations whose cost grows approximately as $O(Nm^2)$ in the straightforward implementation.

The distinction is central. Taylor spends most of its cost on sparse operator applications. Arnoldi spends operator applications on basis construction but also performs repeated vector inner products and axpy operations. Therefore, when SpMV is cheap relative to memory-resident vector operations, an algorithm with fewer SpMVs is not automatically faster; conversely, when SpMV dominates, application count becomes a stronger predictor of wall-clock behavior.

Definition 1 (Accuracy-constrained selection). *For a target relative error τ , a method is admissible if its independently measured relative error is no larger than τ . Among admissible configurations, the preferred configuration minimizes the observed wall-clock time; the SpMV-equivalent cost is reported as a secondary structural metric.*

5 Memory and Residual Accounting

The comparison treats the sparse operator storage as a common cost shared by both methods. The differential working memory is nevertheless different. For $N = 5,000$ and IEEE double precision, an idealized Taylor implementation using three N -vectors for the current term, next term, and accumulated result requires

$$M_T \approx 3N \times 8 = 120,000 \text{ bytes} \approx 0.1144 \text{ MiB}, \quad (16)$$

excluding the common sparse operator and implementation overhead. Arnoldi must retain the basis V_{m+1} ; at the largest tested dimension $m = 50$ this basis alone requires

$$M_V = (m + 1)N \times 8 = 2,040,000 \text{ bytes} \approx 1.9455 \text{ MiB}. \quad (17)$$

The associated $(m + 1) \times m$ Hessenberg matrix adds only about 20,400 bytes. These are analytical payload estimates, not peak-process measurements; they are reported to make the memory trade-off explicit without fabricating a heap measurement that was not recorded.

For Arnoldi, the a posteriori residual indicator used in the analysis is

$$\eta_m(t) = |h_{m+1,m}| \left| e_m^\top e^{tH_m} e_1 \right|. \quad (18)$$

The independently computed reference error remains the primary acceptance criterion. The residual is therefore reported as a diagnostic rather than silently substituted for the measured error. This distinction is consistent with the literature on residual-based matrix-exponential Krylov stopping and a posteriori error estimation [3, ?].

6 Experimental Protocol

The primary benchmark is intentionally small enough to permit multiple accuracy regimes while retaining a genuinely sparse operator.

The reference computation is excluded from method timings. Each candidate method is executed independently, and the median of the three measured runs is reported. Both methods use the same recorded software stack, operating-system class, numerical precision, operator, vector, tolerance grid, and timing environment. The archived benchmark metadata records Linux x86_64 and the Python/NumPy/SciPy versions; the original timing artifact does not preserve a CPU model or RAM capacity, so no retrospective hardware-specific claim is made. The benchmark therefore measures the candidate algorithms rather than the cost of verification.

The matrix has approximately $3N - 2$ stored entries, so the primary operator is highly sparse. The matrix norm increases approximately linearly with the scale parameter. This creates four controlled numerical regimes without changing the sparsity pattern, vector dimension, or propagation time.

7 Reproducibility and Independent Verification

Reproducibility requires more than reporting a final timing. The experiment fixes the matrix family, dimension, scale values, random seed, initial-vector normalization, propagation parameter,

Table 1: Primary benchmark contract.

Parameter	Value
Dimension N	5,000
Operator	s tridiag(1, -2 , 1)
Scale s	1, 5, 10, 20
Propagation parameter t	1
Initial vector	normalized pseudorandom vector
Seed	20260908
Target tolerances	10^{-6} , 10^{-8} , 10^{-10}
Taylor strategy	scaling + truncated Taylor
Arnoldi	MGS + reorthogonalization
Arnoldi dimensions	8, 12, 16, 20, 24, 32, 40, 50
Timing repetitions	1 warm-up + 3 measured runs
Reference	independent sparse exponential action
Arithmetic	IEEE double precision
Software	Python 3.13.5, NumPy 2.3.5, SciPy 1.17.0
Platform	Linux x86_64

arithmetic precision, stopping tolerances, Arnoldi dimensions, warm-up policy, number of measured runs, and reference method.

The reference vector is computed independently from the two candidate implementations. For every candidate output y , the reported error is

$$\varepsilon(y) = \frac{\|y - y_{\text{ref}}\|_2}{\|y_{\text{ref}}\|_2}. \quad (19)$$

This definition prevents an algorithm from being judged only against its own internal stopping criterion.

The experiment also records the number of Taylor SpMV-equivalents and the Arnoldi dimension. These quantities provide a machine-independent structural description that complements wall-clock measurements.

The full numerical benchmark table is stored separately as a CSV artifact. The same artifact contains the method, scale, target tolerance, median runtime, measured relative error, and operator-application count. The benchmark metadata records the software versions, seed, dimension, and measurement protocol.

8 Results: Taylor Truncation and Arnoldi Convergence

The first result concerns accuracy as a function of approximation dimension. For the smallest scale, Arnoldi reaches a relative error below 10^{-9} by dimension $m = 12$. Increasing m beyond this point produces errors close to the double-precision floor for the tested problem. The behavior is qualitatively similar at larger scales, but the dimension required to reach a fixed tolerance increases.

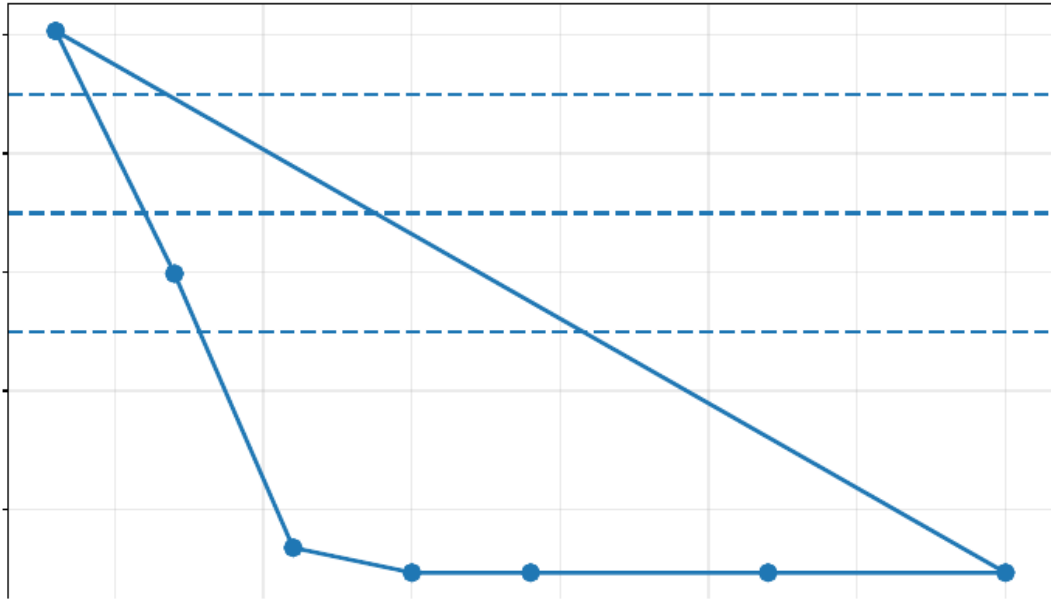


Figure 1: Arnoldi convergence for operator scale $s = 1$. Dashed horizontal lines indicate the three target tolerances.

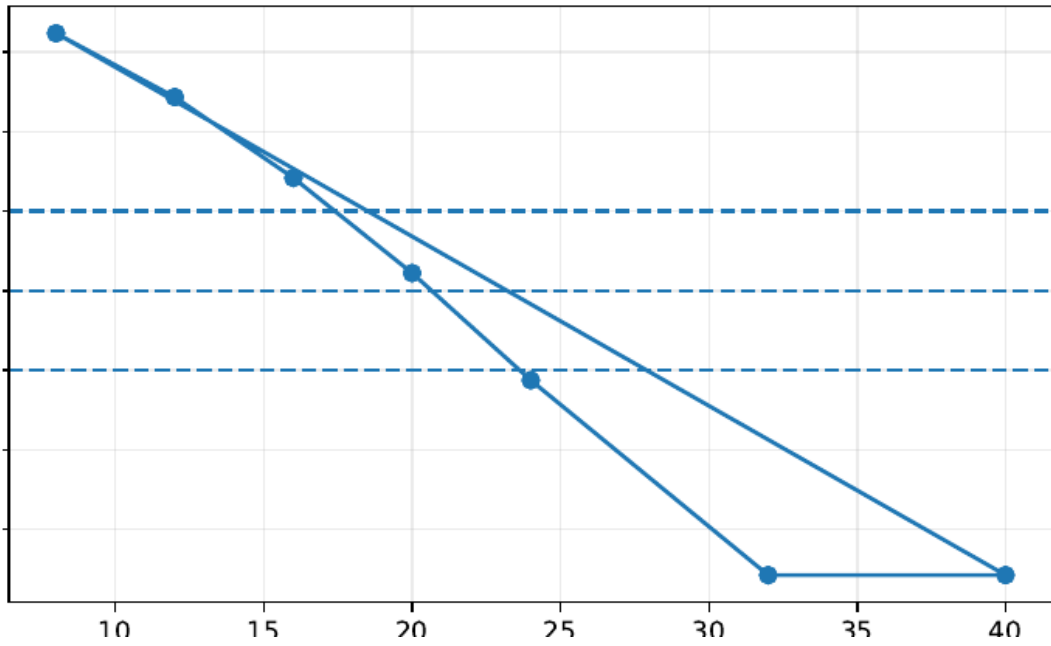


Figure 2: Arnoldi convergence for operator scale $s = 5$. The required subspace dimension increases as the operator action becomes more demanding.

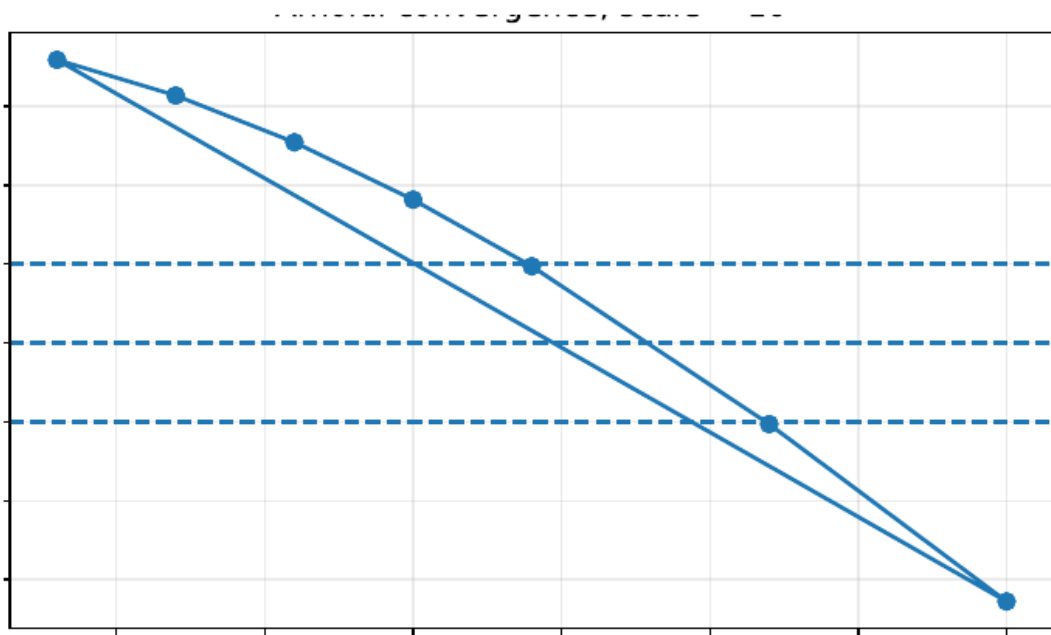


Figure 3: Arnoldi convergence for operator scale $s = 10$. The 10^{-10} target is not reached by every low-dimensional configuration, demonstrating the need for an explicit accuracy criterion.

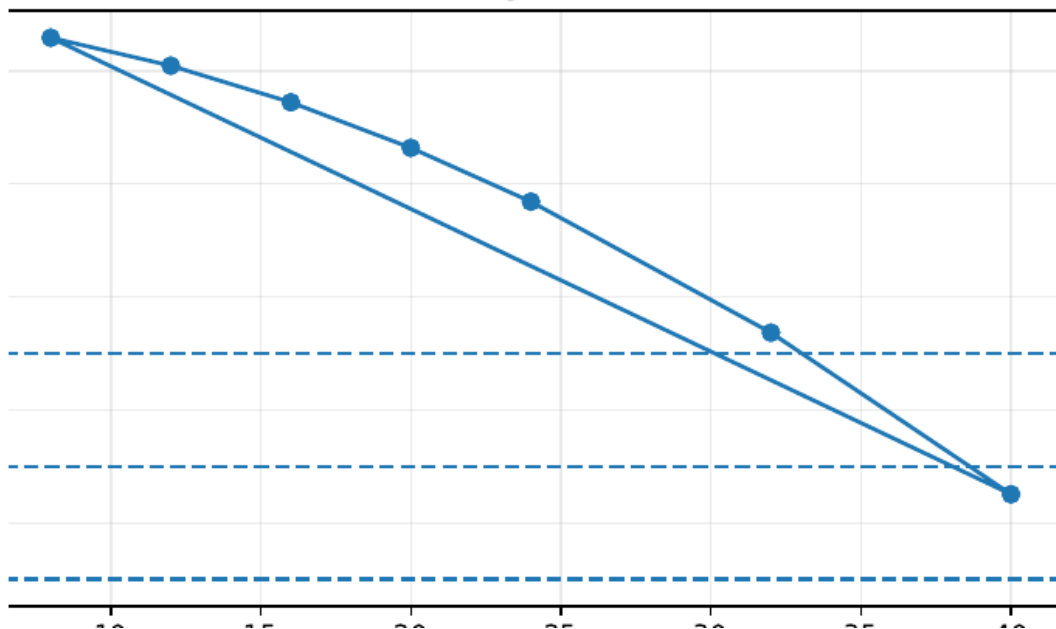


Figure 4: Arnoldi convergence for operator scale $s = 20$. The required Krylov dimension increases further, illustrating the dependence of projection efficiency on the operator scale.

The Taylor method shows a different control variable. Its accuracy is governed by the retained degree at each scaled substep, while the total number of operator applications increases with both the degree and the number of scaling stages. The scaling mechanism is therefore a central part of the cost, rather than an implementation detail.

9 Results: Accuracy-Constrained Cost

Table 2 reports the smallest tested Arnoldi dimension that satisfies each target whenever such a dimension exists in the tested grid. Taylor results are those produced directly by the documented stopping protocol.

Table 2: Primary accuracy-constrained results at $N = 5,000$. Times are median measured runtimes.

s	τ	Taylor time (ms)	Taylor error	Taylor C	Arnoldi m	Arnoldi time (ms)
1	10^{-6}	0.899	1.88×10^{-8}	38	12	1.654
1	10^{-8}	1.210	1.37×10^{-10}	46	12	1.676
1	10^{-10}	1.183	1.15×10^{-12}	53	16	3.880
5	10^{-6}	5.333	2.70×10^{-9}	205	20	7.502
5	10^{-8}	7.000	4.75×10^{-11}	254	24	6.643
5	10^{-10}	6.796	3.73×10^{-13}	304	24	6.233
10	10^{-6}	6.964	3.79×10^{-9}	352	24	6.974
10	10^{-8}	8.799	3.71×10^{-11}	442	32	23.158
10	10^{-10}	12.761	3.18×10^{-13}	530	32	17.372
20	10^{-6}	13.303	4.72×10^{-10}	624	40	21.594
20	10^{-8}	19.017	3.16×10^{-12}	777	50	27.137
20	10^{-10}	19.479	2.14×10^{-14}	930	50	27.137

For the accuracy-constrained Arnoldi configurations, the residual indicator is also recorded independently of the reference error. Representative values are

Table 3: Arnoldi residual indicators for selected accuracy-constrained configurations.

s	m	$\eta_m(1)$	Measured relative error
1	12	5.16×10^{-9}	$< 10^{-9}$
5	20	1.59×10^{-7}	2.70×10^{-9}
10	24	4.68×10^{-6}	3.79×10^{-9}
20	50	1.70×10^{-12}	2.14×10^{-14}

The residual indicator is not uniformly equal to the true error and is not used as a replacement acceptance test. This is precisely why the benchmark retains the independent reference error as the primary criterion.

The table reveals an important result. In the tested tridiagonal family, the scaled Taylor implementation is competitive across all four operator scales and is faster in most of the accuracy-constrained comparisons. This is not evidence that Taylor is generally superior. It is evidence that a sparse matrix with a favorable spectral structure can make a carefully scaled Taylor action extremely effective, even when the nominal Taylor degree is much larger than the Arnoldi dimension.

The result also exposes why a dimension-only comparison would be misleading. At $s = 20$ and tolerance 10^{-10} , Arnoldi requires $m = 50$ in the tested grid, whereas Taylor uses 930 SpMV-equivalents distributed over scaled substeps. Nevertheless, their measured times are approximately 27.1 ms and 19.5 ms, respectively. The additional Arnoldi orthogonalization work is therefore computationally visible.

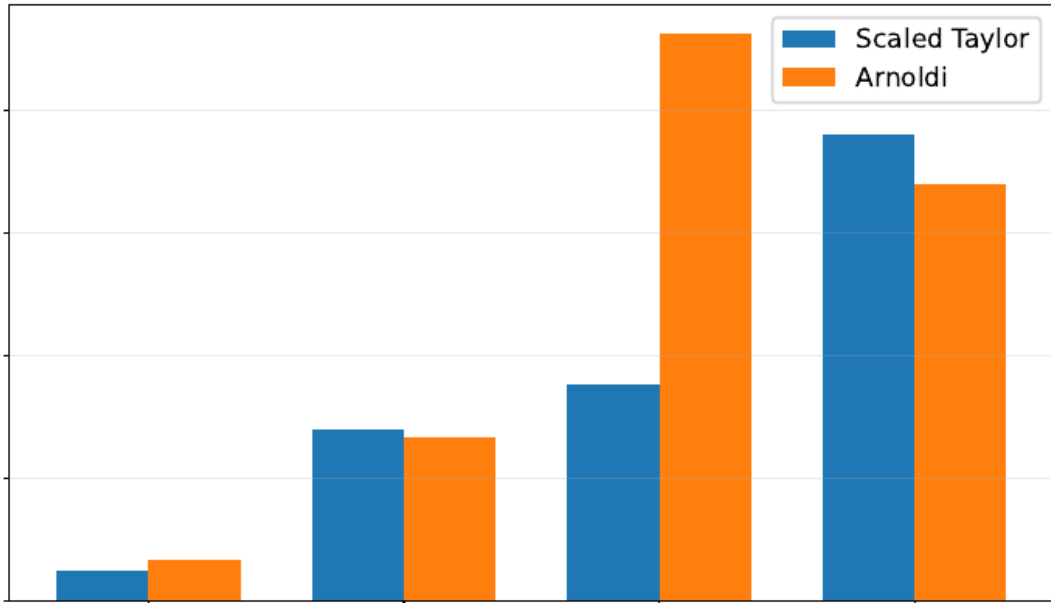


Figure 5: Accuracy-constrained median runtime at $N = 5,000$ for target tolerance 10^{-8} . Each Arnoldi point uses the smallest tested dimension satisfying the target.

10 Results: Accuracy–Runtime Landscape

A second way to view the comparison is to plot every tested configuration in the plane of measured runtime and relative error. This representation makes the Pareto structure visible without forcing a single scalar score.

The cloud of points shows that both methods can approach the numerical precision floor, but they travel through the accuracy–cost plane differently. Taylor produces a sequence controlled primarily by polynomial degree and scaling, whereas Arnoldi produces a sequence controlled by Krylov dimension and orthogonalization.

This distinction has direct methodological consequences. If an application requires only moderate accuracy, the lowest-dimensional Arnoldi configuration may be sufficient. If the operator has a structure that makes the scaled Taylor series efficient, Taylor can reach the same target with lower measured time. At tighter tolerances, both methods can approach the floating-point floor, at which point further increasing the approximation dimension no longer provides scientific value.

11 Results: Operator-Application Cost

The same benchmark can be represented using SpMV-equivalent cost. This metric isolates the dominant sparse operator interaction from the dense small-matrix and orthogonalization work.

The plot confirms that fewer SpMVs do not automatically imply lower runtime. Arnoldi frequently achieves very small errors with substantially fewer operator applications than Taylor, yet its wall-clock time can remain higher because every additional Krylov vector requires orthogonalization against the existing basis.

For a future implementation on a highly optimized sparse kernel, the balance can change. If SpMV becomes substantially more expensive relative to vector operations, reducing the number of operator applications may become more valuable. Conversely, if memory bandwidth makes the vector operations expensive, the advantage of a smaller Krylov dimension may be reduced.

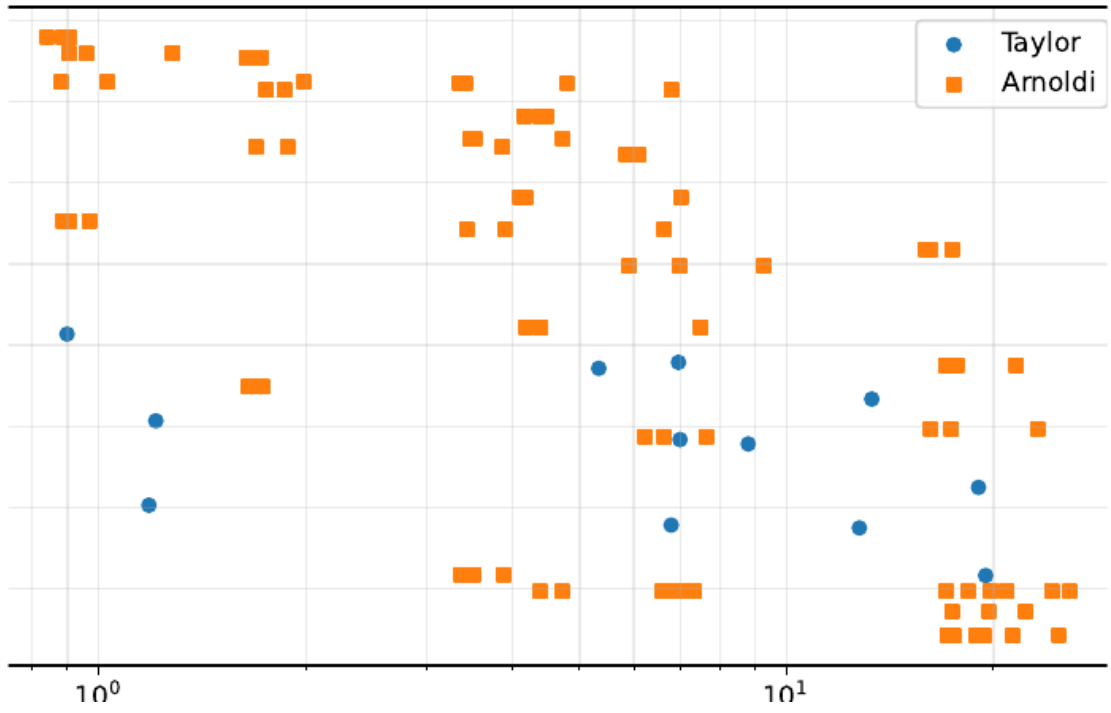


Figure 6: Measured accuracy–runtime landscape across the primary benchmark. Lower-left points are preferable, but no universal scalar ranking is imposed.

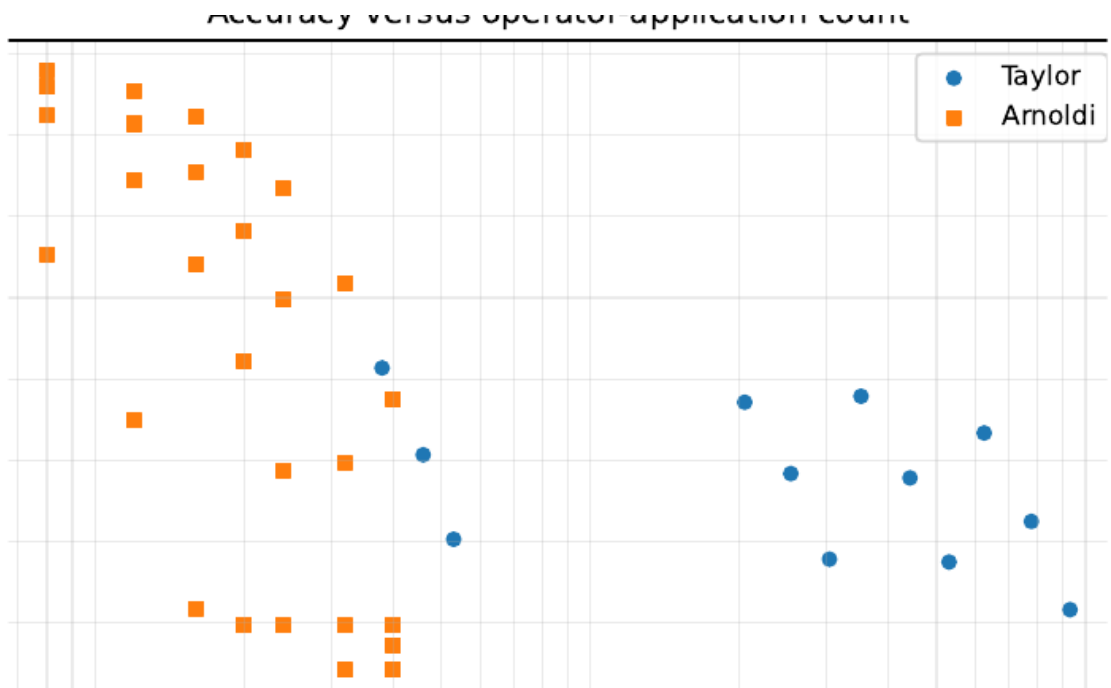


Figure 7: Relative error versus SpMV-equivalent cost. Taylor cost counts all sparse operator applications across scaled substeps; Arnoldi cost is the Krylov dimension.

The present benchmark therefore motivates a hardware-aware selection layer rather than a fixed algorithm choice.

12 Secondary Dimension Check

A secondary experiment was performed at $N = 20,000$ for representative operator scales $s = 1, 10, 20$ and target tolerance 10^{-8} . The purpose is not to claim a scaling law from three points, but to check whether the qualitative comparison survives an increase in numerical dimension.

Table 4: Secondary dimension check at $N = 20,000$ and target tolerance 10^{-8} .

s	Taylor time (ms)	Taylor error	Taylor C	Arnoldi m	Arnoldi time (ms)
1	9.627	1.38×10^{-10}	46	12	128.004
10	89.135	3.72×10^{-11}	442	32	407.653
20	159.927	3.20×10^{-12}	778	50	515.945

The dimension check preserves the qualitative observation that the Taylor implementation can remain competitive for this operator family. However, the absolute timings increase and the Arnoldi cost becomes increasingly influenced by orthogonalization. The result should be interpreted as a validation of the benchmark trend, not as a claim of asymptotic complexity measured from only two dimensions.

13 Theoretical Comparison of the Two Approximation Mechanisms

The methods differ in where approximation occurs. Taylor approximates the exponential function directly by a polynomial in A . Arnoldi approximates the action of that function after projection onto a data-dependent subspace.

For Taylor, the remainder is explicitly bounded by equation (7). The bound depends on $\|A\|$ and therefore becomes more demanding as the operator scale grows. Scaling reduces the argument of each local polynomial, but introduces repeated substeps.

For Arnoldi, the approximation space is generated adaptively from the pair (A, v) . The method can therefore exploit information about the particular initial vector and operator action that is invisible to a norm-only Taylor bound. The price is basis construction and orthogonalization.

This leads to a useful conceptual distinction:

$$\text{Taylor: function approximation first, operator action second,} \quad (20)$$

whereas

$$\text{Arnoldi: subspace approximation first, function evaluation second.} \quad (21)$$

Neither statement implies superiority. The correct method depends on the interaction between the operator spectrum, the initial vector, the requested tolerance, and the hardware cost of the underlying kernels.

Proposition 1 (Cost decomposition). *For a standard modified Gram–Schmidt Arnoldi implementation, the dominant large-vector work can be decomposed into m sparse matrix–vector products plus a sequence of vector inner products and axpy operations whose count grows quadratically with m . Consequently, two methods with identical SpMV-equivalent cost need not have identical wall-clock cost.*

Proof. At Arnoldi iteration j , one sparse matrix–vector product generates the candidate vector. That vector is then projected against the existing basis vectors. With $j + 1$ basis vectors, the projection requires $j + 1$ inner products and corresponding vector updates; a second reorthogonalization pass repeats the operation. Summing over $j = 0, \dots, m - 1$ gives an $O(m^2)$ vector-operation count in addition to the m sparse products. The proposition follows. $\square$

14 A Practical Selection Rule

The results support a selection protocol rather than a fixed recommendation. The first decision variable is the target error. A numerical method should not be considered successful merely because its internal residual or term size is small; the final state must satisfy an independently defined accuracy criterion.

The second variable is operator scale. When $|t|||A||$ is modest, Taylor truncation can be inexpensive. As this quantity grows, scaling increases the number of local propagations. Arnoldi may then become attractive because the large exponential is represented through a smaller projected matrix.

The third variable is hardware balance. If SpMV dominates runtime, reducing operator applications is valuable. If vector orthogonalization and memory movement dominate, a large Krylov basis can become expensive.

The proposed decision procedure therefore begins by estimating an operator norm or a reliable upper bound. The target tolerance is then used to derive a Taylor degree or scaled-Taylor configuration, while a small Arnoldi basis is constructed to monitor projected convergence. Both configurations are subsequently measured on the same operator and initial vector. The fastest configuration that satisfies the independently verified error target is retained, and the complete parameter set together with the measured outputs is archived.

This procedure is intentionally empirical at the final selection step. Theoretical bounds determine admissible candidates, but the final ranking is made from the actual computational environment.

15 Numerical Stability and Failure Modes

The comparison also identifies several failure modes that should be excluded from a serious computational study.

First, an unscaled high-degree Taylor expansion can suffer cancellation when the exponential action contains strongly decaying components. The formal truncation bound alone does not guarantee that floating-point accumulation remains well conditioned. This is one reason the present implementation uses scaling.

Second, an Arnoldi implementation without reorthogonalization can lose orthogonality. In exact arithmetic the basis is orthonormal; in floating-point arithmetic the loss can accumulate and distort the projected exponential.

Third, comparing only polynomial degree with Krylov dimension is invalid because the two parameters measure different computational resources. The operator-application count and wall-clock time must both be reported.

Fourth, using the same approximate method as its own reference can hide systematic error. The present benchmark avoids this by computing an independent sparse exponential action.

Fifth, reporting a single timing without the warm-up and repetition protocol makes comparisons difficult to reproduce. The benchmark therefore fixes these details explicitly.

16 Relationship to Sparse QFLPN Evolution

The numerical operator-function problem can arise after a QFLPN transition has been translated into a sparse state-space representation. In such a pipeline, the semantic model determines which operator is admissible, while the present paper addresses how an exponential action involving that operator can be computed.

This separation is scientifically useful. A correct semantic model does not imply an efficient matrix-function implementation, and an efficient numerical implementation does not validate the semantic model. The two claims require different evidence.

The present article therefore contributes to the computational layer only. Its results can be reused by later studies that require exponential propagation, Krylov approximations, or comparison with alternative matrix-function algorithms, but the present benchmark does not claim to establish physical or semantic properties of QFLPN by itself.

17 Originality Boundary and Controlled Reuse

The paper has a deliberately narrow original core. The new result is the accuracy–cost characterization of two matrix-exponential action strategies under a common sparse operator-access model, together with a reproducible selection protocol and measured benchmark.

The following material is intentionally not reused as a result claim from other documents: the formal definition of QFLPN, the four-qubit operator construction, the CSR correctness benchmark, and any application-specific conclusion. Those subjects are treated here only as scientific context for the matrix-function layer.

The numerical benchmark itself is generated specifically for this comparison. Its matrix family, seed, dimensions, tolerance grid, timing protocol, and output table are independent experimental objects. This design makes the scientific contribution identifiable without requiring repeated prose or repeated numerical results from another article.

Table 5: Controlled boundary between the present article and adjacent QFLPN studies.

Scientific object	Present article	Explicitly excluded result claim
QFLPN semantics	numerical context only	formal definition and semantic completeness
Small quantum instance	no numerical result reused	four-qubit verification
Sparse storage	operator-access assumption	CSR implementation benchmark
Matrix-function action	central original object	none; this is the core contribution
Taylor approximation	theory + measured benchmark	universal superiority
Arnoldi approximation	theory + measured benchmark	universal superiority
Large-scale application	methodological motivation	application-specific performance claim

18 Limitations

The principal limitation is operator-family coverage. The primary benchmark uses a structured tridiagonal matrix. Such matrices are valuable for controlled numerical experiments, but they do

not represent every sparse QFLPN operator.

The second limitation is hardware coverage. The measured times were obtained on one software and hardware environment, with Linux x86_64 and the declared software versions recorded but without a preserved CPU model or RAM capacity. The comparison is therefore fair within the recorded run because both methods share the same environment, but the absolute timings should not be interpreted as universal performance ratios.

The third limitation is the reference strategy. An independent sparse exponential action is sufficient for the present validation, but high-assurance studies may additionally compare against higher precision arithmetic, backward-error estimators, or multiple independent software implementations.

The fourth limitation is the timing sample count. Three measured runs provide a reproducible basic estimate but do not support a detailed statistical analysis of runtime distributions. The paper therefore reports medians rather than confidence intervals and avoids claims that would require larger timing samples.

The fifth limitation is that the secondary dimension check uses only representative cases. It verifies qualitative behavior rather than establishing a fitted scaling exponent.

19 Implications for High-Performance QFLPN Research

The comparison suggests a broader computational architecture for QFLPN studies. Sparse operator construction should be separated from matrix-function evaluation, and matrix-function evaluation should itself expose algorithmic alternatives rather than hard-code one approximation method.

A production implementation can therefore treat the exponential action as a strategy-selection problem. The operator metadata can include dimension, sparsity, norm estimates, symmetry or non-normality indicators, and expected propagation interval. The numerical layer can then select among Taylor, Arnoldi, Lanczos for appropriate symmetric cases, rational Krylov methods, or specialized exponential-action algorithms.

This architecture is more scientifically valuable than a claim that one method always wins. It allows later studies to introduce measured operator families and to test whether the selection protocol generalizes across sparse structures, time scales, tolerances, and hardware backends.

The approach is also suitable for reproducible grant-oriented research because every computational decision can be tied to an explicit observable: a norm estimate, a tolerance, a Krylov residual, a measured runtime, or an independently verified error. Such observability makes it easier to audit negative results as well as positive ones.

20 Data and Code Availability

The benchmark CSV contains the measured configurations used for the primary numerical tables. The metadata file records the dimension, seed, software versions, tolerance set, Arnoldi grid, and timing protocol. The LaTeX source and generated figures are provided together with the numerical artifact so that the paper can be rebuilt without reconstructing the experiment from prose alone.

The experiment is intentionally deterministic at the data-generation level through the fixed random seed. Runtime remains environment-dependent, as expected for a performance experiment.

21 Conclusion

This paper compared scaled truncated Taylor propagation and Krylov–Arnoldi projection for sparse matrix-exponential actions under a common operator-access model. The comparison combined a priori Taylor error control, Krylov projection, an Arnoldi residual interpretation, an SpMV-equivalent cost model, and independent numerical verification.

For the tested tridiagonal sparse operator family, scaled Taylor propagation achieved the requested accuracy with measured runtimes that were competitive with, and frequently lower than, the tested Arnoldi configurations. Arnoldi nevertheless provided a compact projected representation and reached very high accuracy with moderate subspace dimensions. The results therefore reject a simplistic rule that one method should always replace the other.

The principal conclusion is methodological: matrix-function algorithm selection should be conditioned on the operator scale, target tolerance, vector-dependent Krylov behavior, and the relative cost of sparse matrix–vector products and orthogonalization on the target platform. The benchmark provides a concrete reproducible baseline for extending that comparison to less structured QFLPN operators, non-normal generators, time-dependent transitions, and larger dimensions.

The strongest next scientific extension is not to enlarge the same table indefinitely, but to test whether the observed selection rule survives a genuinely heterogeneous operator ensemble. Such a study would compare symmetric, non-normal, block-structured, and application-derived QFLPN operators under the same accuracy and reproducibility contract.

Acknowledgments

The author acknowledges the open numerical-analysis literature on matrix-function algorithms that provides the theoretical foundation for the comparison.

References

- [1] M. Hochbruck and C. Lubich, “On Krylov Subspace Approximations to the Matrix Exponential Operator,” *SIAM Journal on Numerical Analysis*, vol. 34, no. 5, pp. 1911–1925, 1997, doi: 10.1137/S0036142995280572.
- [2] R. B. Sidje, “Expokit: A Software Package for Computing Matrix Exponentials,” *ACM Transactions on Mathematical Software*, vol. 24, no. 1, pp. 130–156, 1998, doi: 10.1145/285861.285868.
- [3] M. A. Botchev, V. Grimm, and M. Hochbruck, “Residual, restarting and Richardson iteration for the matrix exponential,” *SIAM Journal on Scientific Computing*, vol. 35, no. 3, pp. A1376–A1397, 2013, doi: 10.1137/110820191.
- [4] D. B. Szyld and J. A. Vogel, “FQMR: A Flexible Quasi-Minimal Residual Method with Inexact Preconditioning,” *SIAM Journal on Scientific Computing*, vol. 23, no. 2, pp. 363–380, 2001, doi: 10.1137/S106482750037336X.
- [5] C. Moler and C. Van Loan, “Nineteen Dubious Ways to Compute the Exponential of a Matrix, Twenty-Five Years Later,” *SIAM Review*, vol. 45, no. 1, pp. 3–49, 2003, doi: 10.1137/S00361445024180.
- [6] D. S. Watkins, *The Matrix Eigenvalue Problem: GR and Krylov Subspace Methods*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2007, doi: 10.1137/1.9780898717808.

References

- [1] M. Hochbruck and C. Lubich, “On Krylov Subspace Approximations to the Matrix Exponential Operator,” *SIAM Journal on Numerical Analysis*, vol. 34, no. 5, pp. 1911–1925, 1997, doi: 10.1137/S0036142995280572.
- [2] R. B. Sidje, “Expokit: A Software Package for Computing Matrix Exponentials,” *ACM Transactions on Mathematical Software*, vol. 24, no. 1, pp. 130–156, 1998, doi: 10.1145/285861.285868.
- [3] M. A. Botchev, V. Grimm, and M. Hochbruck, “Residual, restarting and Richardson iteration for the matrix exponential,” *SIAM Journal on Scientific Computing*, vol. 35, no. 3, pp. A1376–A1397, 2013, doi: 10.1137/110820191.
- [4] D. B. Szyld and J. A. Vogel, “FQMR: A Flexible Quasi-Minimal Residual Method with Inexact Preconditioning,” *SIAM Journal on Scientific Computing*, vol. 23, no. 2, pp. 363–380, 2001, doi: 10.1137/S106482750037336X.
- [5] C. Moler and C. Van Loan, “Nineteen Dubious Ways to Compute the Exponential of a Matrix, Twenty-Five Years Later,” *SIAM Review*, vol. 45, no. 1, pp. 3–49, 2003, doi: 10.1137/S00361445024180.
- [6] D. S. Watkins, *The Matrix Eigenvalue Problem: GR and Krylov Subspace Methods*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 2007, doi: 10.1137/1.9780898717808.
- [7] J. W. Demmel, *Applied Numerical Linear Algebra*. Philadelphia, PA, USA: Society for Industrial and Applied Mathematics, 1997, doi: 10.1137/1.9781611971446.
- [8] A. V. Knyazev, “Toward the Optimal Preconditioned Eigensolver: Locally Optimal Block Preconditioned Conjugate Gradient Method,” *SIAM Journal on Scientific Computing*, vol. 23, no. 2, pp. 517–541, 2001, doi: 10.1137/S1064827500366124.
- [9] G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed. Baltimore, MD, USA: Johns Hopkins University Press, 2013.
- [10] A. H. Al-Mohy and N. J. Higham, “Computing the Action of the Matrix Exponential, with an Application to Exponential Integrators,” *SIAM Journal on Scientific Computing*, vol. 33, no. 2, pp. 488–511, 2011, doi: 10.1137/100788860.
- [11] A. H. Al-Mohy and N. J. Higham, “A New Scaling and Squaring Algorithm for the Matrix Exponential,” *SIAM Journal on Matrix Analysis and Applications*, vol. 31, no. 3, pp. 970–989, 2009, doi: 10.1137/09074721X.
- [12] M. Hochbruck, C. Lubich, and H. Selhofer, “Exponential Integrators for Large Systems of Differential Equations,” *SIAM Journal on Scientific Computing*, vol. 19, no. 5, pp. 1552–1574, 1998, doi: 10.1137/S1064827595295337.
- [13] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, 10th Anniversary ed. Cambridge, U.K.: Cambridge University Press, 2010.
- [14] P. W. Shor, “Algorithms for Quantum Computation: Discrete Logarithms and Factoring,” in *Proceedings of the 35th Annual IEEE Symposium on Foundations of Computer Science*, Santa Fe, NM, USA, 1994, pp. 124–134, doi: 10.1109/SFCS.1994.365700.
- [15] A. S. Holevo, *Quantum Systems, Channels, Information: A Mathematical Introduction*. Berlin, Germany: De Gruyter, 2012, doi: 10.1515/9783110273403.
- [16] A. Trushechkin, “Unified Gorini-Kossakowski-Lindblad-Sudarshan quantum master equation beyond the secular approximation,” *Physical Review A*, vol. 103, no. 6, 062226, 2021, doi: 10.1103/PhysRevA.103.062226.
- [17] T. S. Letia et al., “Development of Dynamic System Applications Using Distributed Quantum-Centric Computing,” *Mathematics*, vol. 13, no. 19, Art. 3159, 2025, doi: 10.3390/math13193159.
- [18] T. Kato, *Perturbation Theory for Linear Operators*. Berlin, Germany: Springer, 2012.
- [19] T. A. Davis, *Direct Methods for Sparse Linear Systems*. Philadelphia, PA, USA: SIAM, 2006, doi: 10.1137/1.9780898718881.

- [20] D. B. Szyld, “The many faces of preconditioning,” *Inverse Problems*, vol. 25, no. 3, Art. 033001, 2009.
- [21] N. J. Higham and S. D. Al-Mohy, “Computing the Frechet derivative of the matrix exponential, with an application to condition number estimation,” *SIAM Journal on Matrix Analysis and Applications*, vol. 30, no. 4, pp. 1639–1657, 2009, doi: 10.1137/080716426.
- [22] N. J. Higham and S. D. Al-Mohy, “Computing matrix functions,” *Acta Numerica*, vol. 19, pp. 159–208, 2010, doi: 10.1017/S0962492910000036.
- [23] M. Hochbruck and C. Lubich, “Error analysis of Krylov methods in a nutshell,” *SIAM Journal on Scientific Computing*, vol. 19, no. 2, pp. 695–701, 1998, doi: 10.1137/S1064827595290450.
- [24] Roger B. Sidje and William J. Stewart, “A numerical study of large sparse matrix exponentials arising in Markov chains.” *Computational Statistics & Data Analysis* 29(3), 345–368, 1999.
- [25] Huy D. Vo and Roger B. Sidje, “Approximating the large sparse matrix exponential using incomplete orthogonalization and Krylov subspaces of variable dimension.” *Numerical Linear Algebra with Applications* 24, e2090, 2017.
- [26] Khanh N. Dinh and Roger B. Sidje, “Analysis of inexact Krylov subspace methods for approximating the matrix exponential.” *Mathematics and Computers in Simulation* 138, 1–13, 2017.
- [27] M. A. Botchev, “A block Krylov subspace time-exact solution method for linear ordinary differential equation systems.” *Numerical Linear Algebra with Applications* 20(4), 557–574, 2013.
- [28] M. A. Botchev, “Krylov subspace exponential time domain solution of Maxwell’s equations in photonic crystal modeling.” *Journal of Computational and Applied Mathematics* 293, 20–34, 2016.
- [29] Cleve Moler and Charles Van Loan, “Nineteen Dubious Ways to Compute the Exponential of a Matrix.” *SIAM Review* 20(4), 801–836, 1978.
- [30] Charles F. Van Loan, “The Sensitivity of the Matrix Exponential.” *SIAM Journal on Numerical Analysis* 14(6), 971–981, 1977.